NAME

rgbgfx — Game Boy graphics converter

SYNOPSIS

rgbgfx [−CmhOuVwXYZ][−v [−v ...]][−a *attrmap* | −A][−B *color*][−b *base_ids*]
[−c *pal_spec*][−−color *when*][−d *depth*][−i *input_tiles*][−L *slice*]
[−l *base_pal*][−N *nb_tiles*][−n *nb_pals*][−o *out_file*]
[−p *pal_file* | −P][−q *pal_map* | −Q][−r *width*][−s *nb_colors*]
[−t *tilemap* | −T][−W *warning*][−x *quantity*] *file*

DESCRIPTION

The **rgbgfx** program converts PNG images into data suitable for display on the Game Boy and Game Boy Color, or vice-versa.

The main function of **rgbgfx** is to divide the input PNG into 8×8 pixel *squares*, convert each of those squares into 1bpp or 2bpp tile data, and save all of the tile data in a file. It also has options to generate a tile map, attribute map, and/or palette set as well; more on that and how the conversion process can be tweaked below.

ARGUMENTS

rgbgfx accepts the usual short and long options, such as `−V` and `−-version`. Options later in the command line override those set earlier, except for when duplicate options are considered an error. Options can be abbreviated as long as the abbreviation is unambiguous: `−−verb` is `−−verbose`, but `−−ver` is invalid because it could also be `−−version`.

Unless otherwise noted, passing '`-`' (a single dash) as a file name makes **rgbgfx** use standard input (for input files) or standard output (for output files). To suppress this behavior, and open a file in the current directory actually called '`-`', pass `./-` instead. Using standard input or output for more than one file in a single command may produce unexpected results.

rgbgfx accepts decimal, hexadecimal, octal, and binary for numeric option arguments. Decimal numbers are written as usual; hexadecimal numbers must be prefixed with either '`$`' or '`0x`'; octal numbers must be prefixed with either '`&`' or '`0o`'; and binary numbers must be prefixed with either '`%`' or '`0b`'. (The prefixes '`$`' and '`&`' will likely need escaping or quoting to avoid being interpreted by the shell.) Leading zeros (after the base prefix, if any) are accepted, and letters are not case-sensitive. For example, all of these are equivalent: '`42`', `042`, `0x2A`, `0X2A`, `0x2a`, `&52`, `0o52`, `0O052`, `0b00101010`, `0B101010`.

The following options are accepted:

`−a` *attrmap*, `−−attr-map` *attrmap*
: Generate an attribute map, which is a file containing tile "attributes". For each square of the input image, its corresponding attribute map byte contains the mirroring bits (if `−m` was specified), the bank bit (see `−N`), and the palette index. See *Pan Docs*: https://gbdev.io/pandocs/Tile_Maps#bg-map-attributes-cgb-mode-only for the individual bytes' format. The output is written just like the tile map (see `−t`), follows the same order (`−Z`), and has the same size.

`−A`, `−−auto-attr-map`
: Same as `−a` *base_path*.attrmap (see "Automatic output paths").

`−B` *color*, `−−background-color` *color*
: Set a background color to be omitted from output. Colors are accepted in `#rgb` or `#rrggbb` format, or as `transparent`. Input tiles which are entirely the specified background color are ignored and will not be output in tile data file. The tilemap, atrribute map, or palette map files *will* use placeholder values where background tiles were. If a background color is specified, it cannot be used within tiles which are not ignored.

`−b` *base_ids*, `−−base-tiles` *base_ids*
: Set the base IDs for tile map output. *base_ids* should be one or two numbers between 0 and 255, separated by a comma; they are for bank 0 and bank 1 respectively. Both default to 0.

`-C`, `--color-curve`
: Modifies the color palettes (whether they are generated from the input image or taken from an input palette specification) with a color curve mimicking the Game Boy Color's screen. This adjusts the *absolute* RGB color values so that the *perceived* colors, when displayed on Game Boy Color hardware (or an emulator with an accurate display filter), will look like the original colors as displayed on a backlit computer screen. Note that GBC displays can look very different depending on the ambient light and their exact hardware model, so this color curve is only a "best effort".

`-c` *`pal_spec`*, `--colors` *`pal_spec`*
: Use the specified color palettes instead of having **`rgbgfx`** automatically determine some. *`pal_spec`* can be one of the following:

  inline palette spec
  : If *`pal_spec`* begins with a hash character '#', it is treated as an inline palette specification. It should contain a comma-separated list of hexadecimal colors, each beginning with a hash. Colors are accepted in `#rgb` or `#rrggbb` format. To leave one or more gaps in the palette, `#none` can be used instead of any color. Palettes must be separated by a colon or semicolon (the latter may require quoting to avoid special handling by the shell), and spaces are allowed around colons, semicolons and commas; trailing commas and semicolons are allowed. See "EXAMPLES" for an example of an inline palette specification.

  embedded palette spec
  : If *`pal_spec`* is the case-insensitive word `embedded`, then the first four colors of the input PNG's embedded palette are used. It is an error if the PNG is not indexed, or if colors other than these 4 are used. (This is different from the default behavior of indexed PNGs, as then unused entries in the embedded palette are ignored, whereas they are not with `-c embedded`).

  DMG palette spec
  : If *`pal_spec`* starts with case-insensitive `dmg=`, then the following two-digit hexadecimal number specifies four grayscale DMG color indexes. The number functions like the DMG's $FF47 **BGP** register (see *Pan Docs*: https://gbdev.io/pandocs/Palettes.html for more information): the low two bits 0-1 specify which gray shade goes in color index 0, the next two bits 2-3 specify which gray shade goes in color index 1, and so on. Gray shade 0 is the lightest (white), 3 is the darkest (black). If *`pal_spec`* is the case-insensitive word `dmg`, then it acts like `dmg=E4`, i.e. the lightest gray will end up in color index 0, and so on. The same gray shade cannot go in two color indexes. To specify a DMG palette, the input PNG must have all its colors in shades of gray, without any transparent colors.

  automatic palette generation
  : If *`pal_spec`* is the case-insensitive word `auto`, then a palette is automatically generated using the procedure described in "PALETTE GENERATION". This is the default behavior if `-c` was not specified.

  external palette spec
  : Otherwise, *`pal_spec`* is assumed to be an external palette specification. The expected format is `format:path`, where *`path`* is a path to a file (`-` is not treated specially), which will be processed according to the *`format`*. See "PALETTE SPECIFICATION FORMATS" for a list of formats and their descriptions.

`--color` *`when`*
: Specify when to highlight warning and error messages with color: `always`, `never`, or `auto`. `auto` determines whether to use colors based on the *NO_COLOR*: https://no-color.org/ or *FORCE_COLOR*: https://force-color.org/ environment variables, or whether the output is to a TTY.

`-d` *depth*, `--depth` *depth*
: Set the bit depth of the output tile data, in bits per pixel (bpp), either 1 or 2 (the default). This changes how tile data is output, and the maximum number of colors per palette (2 and 4 respectively).

`-h`, `--help`
: Print help text for the program and exit. (Help text wraps to the value of the `COLUMNS` environment variable if that is defined as nonzero; or else to the console window width if output is to a TTY.)

`-i` *input_tiles*, `--input-tileset` *input_tiles*
: Use the specified input tiles in addition to having **rgbgfx** automatically determine some. The input tiles will always be first in the `-o` image output, and will always get the first IDs in the `-t` tilemap output. *input_tiles* must contain 1bpp or 2bpp tile data (whichever matches the `-d` option used here), as could be previously generated with the `-o` option.

  If the `-o` option is also specified, then the input tiles will be assigned the first tile IDs, and any tiles from the input image that are not in the input tileset will be assigned subsequent IDs. But if the `-o` option is *not* specified, then the tile map can *only* use tiles from the input tileset. Using `-o` with `-i` is useful if you want to precisely control the tile IDs of its tile map. Using `-i` alone is more useful if you want several images to use a subset of shared tiles.

  If the image will use more than one color palette, it is *strongly* advised to generate the palette set along with the input tile data, and pass `-c gbc:`*input_palette* along with `-i` *input_tiles*. This is because **rgbgfx** might not generate the same palette set for this image as it did for its input tileset.

  See “EXAMPLES” for examples of how to use this option.

  This option is ignored in “REVERSE MODE”.

`-L` *slice*, `--slice` *slice*
: Only process a given rectangle of the image. This is useful for example if the input image is a sheet of some sort, and you want to convert each cel individually. The default is to process the whole image as-is.

  slice must be formatted as *X*,*Y*:*W*,*H*: two comma-separated number pairs, separated by a colon. Whitespace is allowed around all punctuation. The first number pair specifies the X and Y coordinates of the top-left pixel that will be processed (anything above it or to its left will be ignored). The second number pair specifies how many tiles to process horizontally and vertically, respectively.

  `-L` **is ignored in reverse mode**, no padding is inserted.

`-l` *base_pal*, `--base-palette` *base_pal*
: Set the base ID for attribute map and palette map output. *base_pal* should be a number between 0 and 255. It defaults to 0.

`-m`, `--mirror-tiles`
: Deduplicate tiles that are horizontally and/or vertically symmetrical mirror images of each other. Only one of each unique tile will be saved in the tile data file, with mirror images counting as duplicates. Useful with a tile map and attribute map together (see `-a` and `-t`) to keep track of the duplicated tiles and the dimension(s) mirrored. Implies `-u`. Equivalent to `-XY`.

`-N` *nb_tiles*, `--nb-tiles` *nb_tiles*
: Set a maximum number of tiles that can be placed in each VRAM bank. *nb_tiles* should be one or two numbers between 0 and 256, separated by a comma; if the latter is omitted, it defaults to 0. Setting either number to 0 prevents any tiles from being output in that bank.

  If more tiles are generated than can fit in the two banks combined, **rgbgfx** will abort. If `-N` is not specified, no limit will be set on the amount of tiles placed in bank 0, and tiles will not be placed in bank 1.

`-n` *`nb_pals`*, `--nb-palettes` *`nb_pals`*
: Abort if more than *`nb_pals`* palettes are generated. This may not be more than 256.

  Note that attribute map output only has 3 bits for the palette ID, so a limit higher than 8 may yield incomplete data unless relying on a palette map (see `-q`).

`-O`, `--group-outputs`
: Sets the 'base path' to be the output tile data path from `-o` instead of the input image path (see "Automatic output paths").

`-o` *`out_file`*, `--output` *`out_file`*
: Output the tile data in native 2bpp format or in 1bpp (depending on `-d`) to this file.

`-p` *`pal_file`*, `--palette` *`pal_file`*
: Output the image's palette set to this file.

`-P`, `--auto-palette`
: Same as `-p` *`base_path`*.pal (see "Automatic output paths").

`-q` *`pal_map`*, `--palette-map` *`pal_map`*
: Output the image's palette map to this file. This is useful if the input image contains more than 8 palettes, as the attribute map only contains the lower 3 bits of the palette indices.

`-Q`, `--auto-palette-map`
: Same as `-q` *`base_path`*.palmap (see "Automatic output paths").

`-r` *`width`*, `--reverse` *`width`*
: Switches **`rgbgfx`** into "**reverse**" mode. In this mode, instead of converting a PNG image into Game Boy data, **`rgbgfx`** will attempt to reverse the process, and render Game Boy data into an image. See "REVERSE MODE" below for details.

  `width` is the width of the image to generate, in tiles. `-r -0` chooses a width to make the image as square as possible. This is useful if you do not know the original width.

`-s` *`nb_colors`*, `--palette-size` *`nb_colors`*
: Specify how many colors each palette contains, including the transparent one if any. *`nb_colors`* cannot be more than `1 <<` *`depth`* (see `-d`).

`-t` *`tilemap`*, `--tilemap` *`tilemap`*
: Generate a file of tile indices. For each square of the input image, its corresponding tile map byte contains the index of the associated tile in the tile data file. The IDs wrap around from 255 back to 0, and do not include the bank bit; use `-a` for that. Useful in combination with `-u` and/or `-m` to keep track of duplicate tiles.

`-T`, `--auto-tilemap`
: Same as `-t` *`base_path`*.tilemap (see "Automatic output paths").

`-u`, `--unique-tiles`
: Deduplicate identical tiles. Only one of each unique tile will be saved in the tile data file. Useful with a tile map (see `-t`) to k eep track of the duplicated tiles.

  Note that if this option is enabled, no guarantee is made on the order in which tiles are output; while it *should* be consistent across identical runs of a given **`rgbgfx`** release, the same is not true for different releases.

`-V`, `--version`
: Print the version of the program and exit.

`-v`, `--verbose`
: Be verbose. The verbosity level is increased by one each time the flag is specified, with each level including the previous:

1. Print the **rgbgfx** configuration before taking actions.
2. Print a notice before significant actions.
3. Print some of the actions' intermediate results.
4. Print some internal debug information.
5. Print detailed internal information.

The verbosity level does not go past 6.

Note that verbose output is only intended to be consumed by humans, and may change without notice between RGBDS releases; relying on those for scripts is not advised.

`-W` *warning*, `--warning` *warning*
: Set warning flag *warning*. A warning message will be printed if *warning* is an unknown warning flag. See the "DIAGNOSTICS" section for a list of warnings.

`-w`
: Disable all warning output, even when turned into errors.

`-X`, `--mirror-x`
: Deduplicate tiles that are horizontally symmetrical mirror images of each other across the X axis. Implies `-u`.

`-x` *quantity*, `--trim-end` *quantity*
: Do not output the last *quantity* tiles to the tile data file; no other output is affected. This is useful for trimming "filler" / blank squares at the end of an image. If fewer than *quantity* tiles would have been emitted, the file will be empty.

  Note that this is done *after* deduplication if `-u` was enabled, so you probably don't want to use this option in combination with `-u`.

  Also note that the tiles that don't get output in the tile data *do* still count towards `-N`'s limit, since they may get output in the tilemap or attribute map. If you need to omit input tiles from all output in a way that does not count towards `-N`, use `-B` background tiles.

`-Y`, `--mirror-y`
: Deduplicate tiles that are vertically symmetrical mirror images of each other across the Y axis. Implies `-u`.

`-Z`, `--columns`
: Read squares from the PNG in column-major order (column by column), instead of the default row-major order (line by line). This primarily affects tile map and attribute map output, although it may also change generated tile data and palettes.

`@`*at_file*
: Read more options and arguments from a file, as if its contents were given on the command line. Arguments are separated by whitespace or newlines. Lines starting with a hash sign ('#') are considered comments and ignored.

  No shell processing is performed, such as wildcard or variable expansion. There is no support for escaping or quoting whitespace to be included in arguments. The standard '`--`' to stop option processing also disables at-file processing. Note that while '`--`' can be used*inside* an at-file, it only disables option processing within that at-file, and processing continues in the parent scope.

  See "At-files" below for an explanation of how this can be useful.

At-files

In a given project, many images are to be converted with different flags. The traditional way of solving this problem has been to specify the different flags for each image in the Makefile or build script; this can be inconvenient, as it centralizes all those flags away from the images they concern.

To avoid these drawbacks, you can use "at-files": any command-line argument that begins with an at sign ('@') is interpreted as one, as documented above. At-files can be stored right next to the corresponding image, for example:

```
$ rgbgfx -o image.2bpp -t image.tilemap @image.flags image.png
```

This will read additional flags from the file `image.flags`, which could contain, for example, `-b 128` to specify a base offset for the image's tiles. The above command could be generated from the following *make*(1) rule:

```
%.2bpp %.tilemap: %.flags %.png
	rgbgfx -o $*.2bpp -t $*.tilemap @$*.flags $*.png
```

PALETTE SPECIFICATION FORMATS

The following formats are supported:

`act` *Adobe Photoshop color table*: https://www.adobe.com/devnet-apps/photoshop/fileformatashtml/#50577411_pgfId-1070626.

`aco` *Adobe Photoshop color swatch*: https://www.adobe.com/devnet-apps/photoshop/fileformatashtml/#50577411_pgfId-1055819.

`gbc` A GBC palette memory dump, as emitted by **`rgbgfx -p`**. Useful to force several images to share the same palette.

`gpl` *GIMP palette*: https://docs.gimp.org/2.10/en/gimp-concepts-palettes.html.

`hex` A plain text file with one hexadecimal `rrggbb` color per line, ignoring any empty lines.

`png` An image of square color swatches, with each row defining the colors for one palette. Color swatches can be any square size.

`psp` *Paint Shop Pro palette*: https://www.selapa.net/swatches/colors/fileformats.php#psp_pal.

If you wish for another format to be supported, please open an issue (see "BUGS" below) or contact us, and supply a few sample files.

PALETTE GENERATION

`rgbgfx` must generate palettes from the colors in the input image, unless `-c` was used; in that case, the provided palettes will be used. **If the order of colors in the palettes is important to y ou**, for example because you want to use palette swaps, please use `-c` to specify the palette e xplicitly.

Note that if the image contains *any* transparent pixels, color #0 of *all* palettes will be transparent.

After generating palettes, **`rgbgfx`** sorts colors within those palettes using the following rules: delim $$

- If the PNG file internally contains a palette (often dubbed an "indexed" PNG), then colors in each output palette will be sorted according to their order in the PNG's palette. Any unused entries will be ignored, and only the first entry is considered if there are any duplicates. (If you want a given color to appear more than once, or an unused color to appear at all, you should specify the palettes explicitly instead using `-c`; `-c embedded` may be appropriate.)
- Otherwise, if the PNG only contains shades of gray, they will be categorized into as many "bins" as there are colors per palette, and the palette is set to these bins. The lightest gray will end up in bin #0, and so on; this is the same luminance ordering as the RGB method below. This is equivalent to having specified a DMG palette of `-c dmg=E4`. If tw o distinct grays end up in the same bin, the RGB method is used instead.

  Be careful that **`rgbgfx`** is picky about what it considers "grays": the red, green, and blue components of each color must *all* be *exactly* the same.
- If none of the above apply, colors are sorted from lightest (first) to darkest (last). The definition of luminance that **`rgbgfx`** uses is "$2126 times red + 7152 times green + 722 times blue$".

delim off

Note that the "indexed" behavior depends on an internal detail of how the PNG is saved, specifically its `PLTE` chunk. Since few image editors (such as GIMP) expose that detail, this behavior is only kept for compatibility and should be considered deprecated.

It turns out that palette generation is an NP-complete problem known as "pagination", so **rgbgfx** does not attempt to find the optimal solution, but instead uses an "overload-and-remove" heuristic to find a good one in a reasonable amount of time. (There are no guarantees about how this algorithm will generate palettes, apart from the constraints documented above.) It is possible to compute the optimal solution externally (using a solver, for example), and then provide it to **rgbgfx** via `-c`.

OUTPUT FILES

All files output by **rgbgfx** are binary files, and designed to follow the Game Boy and Game Boy Color's native formats. What follows is succinct descriptions of those formats, including **rgbgfx**-specific details. For more complete, beginner-friendly descriptions of the native formats with illustrations, please check out *Pan Docs*: https://gbdev.io/pandocs/Graphics.

Tile data

Tile data is output like a binary dump of VRAM, with no padding between tiles. Each tile is 16 bytes, 2 per row of 8 pixels; the bits of color IDs are split into each byte (or "bitplane"). The leftmost pixel's color ID is stored in the two bytes' most significant bits, and the rightmost pixel's color ID in their least significant bits.

When the bit depth (`-d`) is set to 1, the most significant bitplane (second byte) of each row, being all zeros, is simply not output.

Palette data

Palette data is output like a dump of palette memory. Each color is written as GBC-native little-endian RGB555, with the unused bit 15 set to 0. There is no padding between colors, nor between palettes; however, empty colors in the palettes are output as 0xFFFF. delim $$ For example, if 5 palettes are generated with `-s 4`, the palette data file will be $2 times 4 times 5 = 40$ bytes long, even if some palettes contain less than 3 colors. delim off Note that `-n` only caps how many palettes are generated (and thus this file's size), but fewer may be generated still.

Tile map data

A tile map is an array of tile IDs, with one byte per tile ID. The first byte always corresponds to the ID of the tile in top-left corner of the input image; the second byte is either the ID of the tile to its right (by default), or below it (with `-Z`); and so on, continuing in the same direction. Rows / columns (respectively) are stored consecutively, with no padding.

Attribute map data

Attribute maps mirror the format of tile maps, like on the GBC, especially the order in which bytes are output. The contents of individual bytes follows the GBC's native format:

Bit 7	BG-to-OAM Priority	Set to 0
Bit 6	Vertical Flip	0=Normal, 1=Mirror vertically
Bit 5	Horizontal Flip	0=Normal, 1=Mirror horizontally
Bit 4	Not used	Set to 0
Bit 3	Tile VRAM Bank number	0=Bank 0, 1=Bank 1
Bit 2–0	Background Palette number	BGP0-7

Note that if more than 8 palettes are used, only the lowest 3 bits of the palette ID are output.

Automatic output paths

For convenience, **rgbgfx** provides shortcuts to generate all files in the same directory. This is done by using the uppercase version of a flag (for example, `-A` instead of `-a`). The *base_path* is the input image path (or the output tile data path from `-o`, if `-O` w as given) with its extension, if any, removed.

For example, these two commands are equivalent:

```
$ rgbgfx img/player.png -o build/player.2bpp -P
$ rgbgfx img/player.png -o build/player.2bpp -p img/player.pal
```

And so are these two:

```
$ rgbgfx img/player.png -o build/player.2bpp -O -P
$ rgbgfx img/player.png -o build/player.2bpp -p build/player.pal
```

REVERSE MODE

rgbgfx can produce a PNG image from valid data. This may be useful for ripping graphics, recovering lost source images, etc. An important caveat on that last one, though: the conversion process is **lossy** both ways, so the "reversed" image won't be perfectly identical to the original—but it should be close to a Game Boy's output. (Keep in mind that many of consoles output different colors, so there is no true reference rendering.)

When using reverse mode, make sure to pass the same flags that were given when generating the data, especially `-C`, `-d`, `-N`, `-s`, `-x`, and `-Z`. ""At-files" may help with this". **rgbgfx** will warn about any inconsistencies it detects.

Files that are normally outputs (`-a`, `-p`, `-t`) become inputs, and *file* will be written to instead of read from, and thus needs not exist beforehand. Any of these inputs not passed is assumed to be some default:

palettes	Unspecified palette data makes **rgbgfx** assume DMG (monochrome Game Boy) mode: a single palette of 4 grays. It is possible to pass palettes using `-c` instead of `-p`.
tile data	Tile data must be provided, as there is no reasonable assumption to fall back on.
tile map	A missing tile map makes **rgbgfx** assume that tiles were not deduplicated, and should be laid out in the order they are stored.
attribute map	Without an attribute map, **rgbgfx** assumes that no tiles were mirrored.

DIAGNOSTICS

Warnings are diagnostic messages that indicate possibly erroneous behavior that does not necessarily compromise the conversion process. The following options alter the way warnings are processed.

`-Werror`
: Make all warnings into errors. This can be negated as `-Wno-error` to prevent turning all warnings into errors.

`-Werror=`
: Make the specified warning or meta warning into an error. A warning's name is appended (example: `-Werror=obsolete`), and this warning is implicitly enabled and turned into an error. This can be negated as `-Wno-error=` to prevent turning a specified warning into an error, even if `-Werror` is in effect.

The following warnings are "meta" warnings, that enable a collection of other warnings. If a specific warning is toggled via a meta flag and a specific one, the more specific one takes priority. The position on the command-line acts as a tie breaker, the last one taking effect.

`-Wall`
: This enables warnings that are likely to indicate an error or undesired behavior, and that can easily be fixed.

`-Weverything`
: Enables literally every warning.

The following warnings are actual warning flags; with each description, the corresponding warning flag is included. Note that each of these flags also has a negation (for example, `-Wobsolete` enables the warning that `-Wno-obsolete` disables; and `-Wall` enables every warning that `-Wno-all` disables). The flags are listed alphabetically below.

`-Wembedded`
: Warn when a generated palette is sorted according to the input PNG's embedded palette but `-c embedded` was not provided.

`-Wobsolete`
: Warn when obsolete features are encountered, which have been deprecated and may later be removed. This warning is enabled by default.

`-Wtrim-nonempty`
: Warn when `-x` trims a nonempty tile. An "empty" tile uses entirely color 0 of its palette. This warning is enabled by `-Wall`.

EXAMPLES

The following will only validate the `tileset.png` image (check its size, that all tiles have a suitable amount of colors, etc.), but output nothing:

```
$ rgbgfx src/res/maps/overworld/tileset.png
```

The following will convert the `tileset.png` image using the two given palettes (and only those), and store the generated 2bpp tile data in `tileset.2bpp`, and the attribute map in `tileset.attrmap`.

```
$ rgbgfx -c '#ffffff,#8d05de, #dc7905,#000000; #fff,#8d05de,
#7e0000 , #000' -A -o tileset.2bpp tileset.png
```

The following will deduplicate the tiles in the `title_screen.png` image, keeping only one of each unique tile, and store the generated 2bpp tile data in `title_screen.2bpp`, and the tile map in `title_screen.tilemap`.

```
$ rgbgfx -u title_screen.png -o title_screen.2bpp -t
title_screen.tilemap
```

The following will convert the given inline palette specification to a palette set, and store the palette set in `colors.pal`, without needing an input image.

```
$ rgbgfx -c '#fff,#ff0,#f80,#000' -p colors.pal
```

The following will convert two level images using the same tileset, and error out if any of them contain tiles not in the tileset.

```
$ rgbgfx tileset.png -o tileset.2bpp -O -P
$ rgbgfx level1.png -i tileset.2bpp -c gbc:tileset.pal -t level1.tilemap -a
$ rgbgfx level2.png -i tileset.2bpp -c gbc:tileset.pal -t level2.tilemap -a
```

BUGS

Please report bugs or mistakes in this documentation on *GitHub*: https://github.com/gbdev/rgbds/issues.

SEE ALSO

rgbasm(1), *rgblink*(1), *rgbfix*(1), *rgbds*(7)

The Game Boy hardware reference *Pan Docs*: https://gbdev.io/pandocs/Graphics, particularly the section about graphics.

HISTORY

`rgbgfx` was originally written by stag019 as a program to be packaged in RGBDS. It was later rewritten by ISSOtm, and is now maintained by a number of contributors at https://github.com/gbdev/rgbds.